Chapter 8: Process handling

Process IDs and Job Numbers:

Every command has a process ID/ PID and each job is assigned with a job number.

```
gayat@GSP-HP:~/SummerInternship$ echo $$
18296
gayat@GSP-HP:~/SummerInternship$ jobs
gayat@GSP-HP:~/SummerInternship$ ps
    PID TTY          TIME CMD
  18296 pts/2    00:00:01 bash
  41114 pts/2    00:00:00 ps
```

Echo $$--> to show the shell's PID

Jobs→ to see the active jobs in the current shell

Ps→ shows current processes

Job numbers refer to background processes that are currently running under your shell, while process IDs refer to all processes currently running on the entire system, for all users. The term job basically refers to a command line that was invoked/executed from your shell.

Job Control:

Foreground jobs→ the commands running in the shell, so one command waits for the previous to finish before running.

Background jobs→ those which can be executed in the background, so that the shell is still accessible for other commands to run. Marked by ending the command with &.

If you want some other job, put into the foreground, you need to use the command name, preceded by a per cent sign (%), or you can use its job number, also preceded by %, or its process ID without a per cent sign. If you don't remember which jobs are running, you can use the command jobs to list them

```
gayat@GSP-HP:~$ sleep 60 &
[2] 41822
gayat@GSP-HP:~$ fg %1
sleep 60
```

```
gayat@GSP-HP:~$ jobs
[1]+  Running                 sleep 60 &
```

To end a job, Ctrl Z. You will probably also find it useful to suspend a job and resume it in the background instead of the foreground.

```
[2] 41822
gayat@GSP-HP:~$ fg %1
sleep 60
gayat@GSP-HP:~$ sleep 60
[2]+  Done                    sleep 60
^Z
[1]+  Stopped                 sleep 60
gayat@GSP-HP:~$ bg %1
[1]+ sleep 60 &
```

Ways to refer to bg jobs:

Reference	Background job
%N	Job number *N*
%string	Job whose command begins with *string*
%?string	Job whose command contains *string*

%+	Most recently invoked background job
%%	Same as above
%-	Second most recently invoked background job

SIGNALS:

Keyboard generated signals:

Key Combo	Signal Name	Meaning
Ctrl+C	SIGINT	Interrupt — kill the process
Ctrl+Z	SIGTSTP	Suspend — pause the process
Ctrl+\	SIGQUIT	Quit — kill and create core dump (rarely used)

Kill command:

Send signals manually to kill. Kill sends the terminate command to the bash and that can kill any process, more like the ctrl+c

```
gayat@GSP-HP:~$ sleep 100 &
[2] 1007
gayat@GSP-HP:~$ kill %2
[2]-  Terminated              sleep 100
gayat@GSP-HP:~$
```

```
gayat@GSP-HP:~$ sleep 100 &
[2] 1498
gayat@GSP-HP:~$ kill -KILL %2
[2]-  Killed                  sleep 100
gayat@GSP-HP:~$ sleep 100 &
[2] 1586
gayat@GSP-HP:~$ kill -QUIT %2
gayat@GSP-HP:~$ kill %2
[2]-  Quit                    (core dumped) sleep 100
bash: kill: %2: no such job
```

Kill -KILL %1 → gives the message KILLED

Kill -QUIT %1→ gives the message EXIT

Ps command:

Gives you the list of all processes with the PID

Just ps→ gives the list of current processes

ps -aux→ detailed list of all processes

ps -u $USER→ just the name of all processes

Example of a practical use:

```
gayat@GSP-HP:~$ ps aux | grep sleep
gayat        701  0.0  0.0   3124  1164 pts/4    T    06:48   0:00 sleep 100
gayat       3513  0.0  0.0   4088  2028 pts/4    S+   06:59   0:00 grep --color=auto sleep
```

Trap:

To handle signals like SIGINT, SIGOUT, EXIT etc

THE SYNTAX:

```
trap 'commands' SIGNAL
```

20250623-0705-57.5427773.mp4 Example of trap

Functions and trap command:

--to call a function: you can write a function and then call this function to run automatically

PID variables and temp file:

```
gayat@GSP-HP:~$ echo $$
5951
gayat@GSP-HP:~$ echo $!

gayat@GSP-HP:~$
```

```
echo $$   # PID of current shell
echo $!   # PID of last background command
```

To reset traps:

```
trap - SIGINT
```

This untraps the signal and returns to normal behaviour.

Disown: This allows a background job to remain independent, so it can still run even after the terminal is closed.

```
gayat@GSP-HP:~$ sleep 90 &
[1] 8303
gayat@GSP-HP:~$ disown
```

COROUTINES:
When two (or more) processes are explicitly programmed to run simultaneously and possibly communicate with each other, they are referred to as coroutines. **Coroutines** aren't like full-blown threads, but you can simulate them using background processes and wait. Pipeline is an example of coroutines, where output of one command becomes the input of another command simultaneously!

So using commands like wait, you can see if a script doesn't finish before the other.

```
gayat@GSP-HP:~$ sleep 5 &
sleep 3 &
wait
echo "Both the sleeps are done!"
[1] 10577
[2] 10578
[1]-  Done                    sleep 5
[2]+  Done                    sleep 3
Both the sleeps are done!
gayat@GSP-HP:~$
```

Advantages and disadvantages of Coroutines:

A:

Simple way to achieve parallelism

Saves time in I/O heavy commands and tasks

D:

No shared memory

Limited communication

Error handling is difficult

Parallelisation: **Parallelisation** means running **multiple commands at the same time** instead of one after the other. Uses & and wait command to run multiple commands simultaneously.

```
gayat@GSP-HP:~$ sleep 2 &
sleep 3 &
sleep 1 &

wait
echo "All tasks done!"
[1] 10732
[2] 10733
[3] 10734
[1]   Done                    sleep 2
[3]+  Done                    sleep 1
[2]+  Done                    sleep 3
All tasks done!
```

- sleep 2, sleep 3, and sleep 1 run **at the same time**.
- Without wait, the script may exit before they finish.
- With wait, Bash waits for **all** to finish before continuing

SUBSHELLS:
A child shell created by Bash to run a command or group of commands in isolation.

Subshell inheritance:

The subshell gets the current directory, environmental variables, standard input, output and error, file descriptors and even signals to be ignored from the parent shell.

```
gayat@GSP-HP:~$ name=Alicia
(
        name=Bob
        echo "Inside: $name"
)
echo "Outside: $name"
Inside: Bob
Outside: Alicia
gayat@GSP-HP:~$
```

Any cd in a subshell is temporary

Nested subshell→ a shell within a shell

```
(
  echo "Outer"
  (
    echo "Inner"
  )
)
```

Process substitution:

This lets you treat the output of a command like a file, even when the command isn't done running. The syntax is as shown: `<(command)`

```
gayat@GSP-HP:~$ diff <(ls SummerInternship/) <(ls snap/)
1,35c1
< 123.txt
< 5
< 5678.txt
< A.sh
< END
< Untitled - Copy.txt:Zone.Identifier
< age.sh
< albums
< and
< cd ( ).sh
< comp.sh
< echo.sh
< file.txt
< file1.txt
< file2.txt
< file3.txt
< file4.txt
< file5.txt
< fw.sh
< gl.sh
< gl2.sh
< grep.sh
< hello.txt
< mouth,
< myfilecreated.txt
< new.txt
< pass.sh
< pushd ().sh
< stg1="Hello".sh
< test1
< test2
```

What this does:

- Runs ls dir1 and ls dir2 **in the background**.
- Feeds their output to diff **as if they were files**.

It's like

diff file1 file2